

Лабораторная работа №7

Введение в работу с данными

Доберштейн А. С.

Содержание

1	Цель работы	5
2	Выполнение лабораторной работы	6
3	Выводы	38

Список иллюстраций

2.1	Считывание данных	6
2.2	Считывание данных	7
2.3	Считывание данных	7
2.4	Считывание данных	8
2.5	Считывание данных	8
2.6	Запись данных в CSV-файл	9
2.7	Словари	9
2.8	Словари	10
2.9	DataFrames	10
2.10	DataFrames	11
2.11	DataFrames	11
2.12	RDatasets	12
2.13	RDatasets	12
2.14	Работа с переменными отсутствующего типа (Missing Values)	13
2.15	Работа с переменными отсутствующего типа (Missing Values)	13
2.16	Работа с переменными отсутствующего типа (Missing Values)	13
2.17	Работа с переменными отсутствующего типа (Missing Values)	14
2.18	FileIO	14
2.19	FileIO	15
2.20	Кластеризация данных. Метод k-средних	16
2.21	Кластеризация данных. Метод k-средних	16
2.22	Кластеризация данных. Метод k-средних	17
2.23	Кластеризация данных. Метод k-средних	17
2.24	Кластеризация данных. Метод k-средних	18
2.25	Кластеризация данных. Метод k-средних	18
2.26	Кластеризация данных. Метод k-средних	19
2.27	Кластеризация данных. Метод k-средних	20
2.28	Кластеризация данных. Метод k-средних	21
2.29	Кластеризация данных. Метод k-средних	22
2.30	Кластеризация данных. Метод k ближайших соседей	23
2.31	Кластеризация данных. Метод k ближайших соседей	23
2.32	Кластеризация данных. Метод k ближайших соседей	24
2.33	Обработка данных. Метод главных компонент	24
2.34	Обработка данных. Метод главных компонент	25
2.35	Обработка данных. Метод главных компонент	25
2.36	Обработка данных. Линейная регрессия	26
2.37	Обработка данных. Линейная регрессия	26

2.38	Обработка данных. Линейная регрессия	27
2.39	Обработка данных. Линейная регрессия	27
2.40	Задание для самостоятельного выполнения №1	28
2.41	Задание для самостоятельного выполнения №1	28
2.42	Задание для самостоятельного выполнения №1	29
2.43	Задание для самостоятельного выполнения №1	30
2.44	Задание для самостоятельного выполнения №2	30
2.45	Задание для самостоятельного выполнения №2	31
2.46	Задание для самостоятельного выполнения №2	31
2.47	Задание для самостоятельного выполнения №2	32
2.48	Задание для самостоятельного выполнения №2	32
2.49	Задание для самостоятельного выполнения №2	32
2.50	Задание для самостоятельного выполнения №3	33
2.51	Задание для самостоятельного выполнения №3	34
2.52	Задание для самостоятельного выполнения №3	34
2.53	Задание для самостоятельного выполнения №3	35
2.54	Задание для самостоятельного выполнения №3	35
2.55	Задание для самостоятельного выполнения №3	36
2.56	Задание для самостоятельного выполнения №3	36
2.57	Задание для самостоятельного выполнения №3	37

1 Цель работы

Основная цель работы — освоить специализированные пакеты Julia для обработки данных.

2 Выполнение лабораторной работы

Повторим примеры из лабораторной работы. (рис. 2.1 - 2.39).

```
# Обновление окружения:
using Pkg
Pkg.update
# Установка пакетов:
using Pkg
for p in ["CSV", "DataFrames", "RDatasets", "FileIO"]
Pkg.add(p)
end

[36m[1m Project[22m[39m No packages added to or removed from `~/julia/environm

[5]:
using CSV, DataFrames, DelimitedFiles
```

Рис. 2.1: Считывание данных

```
[7]:
# Считывание данных и их запись в структуру:
P = CSV.File("programminglanguages.csv") |> DataFrame

[7]:
73x2 DataFrame                                48 rows omitted
  Row  year  language
   Int64 String31
-----
  1  1951  Regional Assembly Language
  2  1952  Autocode
  3  1954  IPL
  4  1955  FLOW-MATIC
  5  1957  FORTRAN
  6  1957  COMTRAN
  7  1958  LISP
  8  1958  ALGOL 58
  9  1959  FACT
 10  1959  COBOL
 11  1959  RPG
 12  1962  APL
 13  1962  Simula
   ⋮      ⋮
 62  2003  Scala
 63  2005  F#
 64  2006  PowerShell
```

Рис. 2.2: Считывание данных

```
[8]:
# Функция определения по названию языка программирования года его создания:
function language_created_year(P, language::String)
  loc = findfirst(P[:,2].==language)
  return P[loc,1]
end
# Пример вызова функции и определение даты создания языка Python:
language_created_year(P, "Python")

[8]:
1991
```

Рис. 2.3: Считывание данных

```
[9]:
# Пример вызова функции и определение даты создания языка Julia:
language_created_year(P, "Julia")

[9]:
2012

[10]:
language_created_year(P, "julia")

MethodError: no method matching getindex(::DataFrame, ::Nothing, ::Int64) ...

[11]:
# Функция определения по названию языка программирования
# года его создания (без учёта регистра):
function language_created_year_v2(P, language::String)
    loc = findfirst(lowercase.(P[:,2]).==lowercase.(language))
    return P[loc,1]
end
# Пример вызова функции и определение даты создания языка julia:
language_created_year_v2(P, "julia")

[11]:
2012
```

Рис. 2.4: Считывание данных

```
[12]:
# Построчное считывание данных с указанием разделителя:
Tx = readlm("programminglanguages.csv", ',')

[12]:
74x2 Matrix{Any}:
  "year"  "language"
1951     "Regional Assembly Language"
1952     "Autocode"
1954     "IPL"
1955     "FLOW-MATIC"
1957     "FORTRAN"
1957     "COMTRAN"
1958     "LISP"
1958     "ALGOL 58"
1959     "FACT"
1959     "COBOL"
1959     "RPG"
1962     "APL"
⋮
2003     "Scala"
2005     "F#"
2006     "PowerShell"
2007     "Clojure"
2009     "Go"
2010     "Rust"
2011     "Dart"
2011     "Kotlin"
2011     "Red"
2011     "Elixir"
2012     "Julia"
2014     "Swift"
```

Рис. 2.5: Считывание данных


```

[13]:
# Запись данных в CSV-файл:
CSV.write("programming_languages_data2.csv", P)

[13]:
"programming_languages_data2.csv"

[14]:
# Пример записи данных в текстовый файл с разделителем ',':
writedlm("programming_languages_data.txt", Tx, ',')

[15]:
# Пример записи данных в текстовый файл с разделителем '-':
writedlm("programming_languages_data2.txt", Tx, '-')

[16]:
# Построчное считывание данных с указанием разделителя:
P_new_delim = readdlm("programming_languages_data2.txt", '-')

[16]:
74x2 Matrix{Any}:
  "year"  "language"
1951      "Regional Assembly Language"
1952      "Autocode"
1954      "IPL"
1955      "FLOW-MATIC"
1957      "FORTRAN"
1957      "COMTRAN"
1958      "LISP"
1958      "ALGOL 58"
1959      "FACT"
1959      "COBOL"
1959      "RPG"
1962      "APL"
⋮
2003      "C++"

```

Рис. 2.6: Запись данных в CSV-файл

```

[17]:
# Инициализация словаря:
dict = Dict{Integer,Vector{String}}()

[17]:
Dict{Integer, Vector{String}}()

[18]:
# Инициализация словаря:
dict2 = Dict()

[18]:
Dict{Any, Any}()

```

Рис. 2.7: Словари

```

# Заполнение словаря данными:
for i = 1:size(P,1)
    year,lang = P[i,:];
    if year in keys(dict)
        dict[year] = push!(dict[year],lang)
    else
        dict[year] = [lang]
    end
end
end

[20]:

# Пример определения в словаре языков программирования, созданных в 2003 году:
dict[2003]

[20]:

2-element Vector{String}:
"Groovy"
"Scala"

```

Рис. 2.8: Словари

```

[21]:

# Подгружаем пакет DataFrames:
using DataFrames
# Задаём переменную со структурой DataFrame:
df = DataFrame(year = P[:,1], language = P[:,2])

[21]:

73×2 DataFrame                                     48 rows omitted
  Row  year  language
   Int64  String31
1    1951  Regional Assembly Language
2    1952  Autocode
3    1954  IPL
4    1955  FLOW-MATIC
5    1957  FORTRAN
6    1957  COMTRAN
7    1958  LISP
8    1958  ALGOL 58
9    1959  FACT
10   1959  COBOL
11   1959  RPG
12   1962  APL
13   1962  Simula
⋮      ⋮      ⋮
62   2003  Scala
63   2005  F#

```

Рис. 2.9: DataFrames

```
# Вывод всех значения столбца year:
df[:,year]

[22]:
73-element Vector{Int64}:
 1951
 1952
 1954
 1955
 1957
 1957
 1958
 1958
 1959
 1959
 1959
 1962
 1962
  ⋮
2003
2005
2006
2007
2009
2010
2011
2011
2011
2011
2012
2014
```

Рис. 2.10: DataFrames

```
[23]:
# Получение статистических сведений о фрейме:
describe(df)

[23]:
2×7 DataFrame
 Row variable  mean    min    median  max  nmissing  eltype
 Symbol Union... Any Union... Any Int64  DataType
 1  year      1982.99 1951    1986.0 2014         0 Int64
 2 language      ALGOL 58      dBase III         0 String31
```

Рис. 2.11: DataFrames

```
# Подгружаем пакет RDatasets:
using RDatasets
# Задаём структуру данных в виде набора данных:
iris = dataset("datasets", "iris")
```

[24]:

150×5 DataFrame 125 rows omitted

Row	SepalLength	SepalWidth	PetalLength	PetalWidth	Species
	Float64	Float64	Float64	Float64	Cat...
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3.0	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
5	5.0	3.6	1.4	0.2	setosa
6	5.4	3.9	1.7	0.4	setosa
7	4.6	3.4	1.4	0.3	setosa
8	5.0	3.4	1.5	0.2	setosa
9	4.4	2.9	1.4	0.2	setosa
10	4.9	3.1	1.5	0.1	setosa
11	5.4	3.7	1.5	0.2	setosa
12	4.8	3.4	1.6	0.2	setosa
13	4.8	3.0	1.4	0.1	setosa
⋮	⋮	⋮	⋮	⋮	⋮
139	6.0	3.0	4.8	1.8	virginica
140	6.9	3.1	5.4	2.1	virginica

Рис. 2.12: RDatasets

```
[25]:
# Определения типа переменной:
typeof(iris)
```

[25]:

DataFrame

```
[26]:
describe(iris)
```

[26]:

5×7 DataFrame

Row	variable	mean	min	median	max	nmissing	eltype
	Symbol	Union...	Any	Union...	Any	Int64	DataType
1	SepalLength	5.84333	4.3	5.8	7.9	0	Float64
2	SepalWidth	3.05733	2.0	3.0	4.4	0	Float64
3	PetalLength	3.758	1.0	4.35	6.9	0	Float64
4	PetalWidth	1.19933	0.1	1.3	2.5	0	Float64
5	Species		setosa		virginica	0	CategoricalValue{String, UInt8}

Рис. 2.13: RDatasets

```
[27]:
# Отсутствующий тип:
a = missing
typeof(a)

[27]:
Missing

[28]:
# Пример операции с переменной отсутствующего типа:
a + 1

[28]:
missing
```

Рис. 2.14: Работа с переменными отсутствующего типа (Missing Values)

```
[29]:
# Определение перечня продуктов:
foods = ["apple", "cucumber", "tomato", "banana"]
# Определение калорий:
calories = [missing, 47, 22, 105]

[29]:
4-element Vector{Union{Missing, Int64}}:
 missing
    47
    22
   105

[30]:
# Определение типа переменной:
typeof(calories)

[30]:
Vector{Union{Missing, Int64}} (alias for Array{Union{Missing, Int64}, 1})
```

Рис. 2.15: Работа с переменными отсутствующего типа (Missing Values)

```
[31]:
# Подключаем пакет Statistics:
using Statistics
# Определение среднего значения:
mean(calories)

[31]:
missing

[32]:
# Определение среднего значения без значений с отсутствующим типом:
mean(skipmissing(calories))

[32]:
58.0
```

Рис. 2.16: Работа с переменными отсутствующего типа (Missing Values)

```
[33]:
# Задание сведений о ценах:
prices = [0.85,1.6,0.8,0.6]
# Формирование данных о калориях:
dataframe_calories = DataFrame(item=foods,calories=calories)
# Формирование данных о ценах:
dataframe_prices = DataFrame(item=foods,price=prices)
# Объединение данных о калориях и ценах:
DF = innerjoin(dataframe_calories,dataframe_prices,on=:item)
```

```
[33]:
4×3 DataFrame
Row  item      calories  price
   String  Int64?  Float64
1  apple      missing    0.85
2  cucumber      47      1.6
3  tomato       22      0.8
4  banana      105      0.6
```

Рис. 2.17: Работа с переменными отсутствующего типа (Missing Values)

```
[34]:
# Подключаем пакет FileIO:
using FileIO
```

```
[35]:
# Подключаем пакет ImageIO:
import Pkg
Pkg.add("ImageIO")
```

⌕[32m⌕[1m Resolving⌕[22m⌕[39m package versions... ●●●

Рис. 2.18: FileIO

[illegible]

Рис. 2.19: FileIO

```
[38]:
# Загрузка пакетов:
import Pkg
Pkg.add("DataFrames")
Pkg.add("Statistics")
using DataFrames
using CSV
import Pkg
Pkg.add("Plots")

[32m[1m Resolving[22m[39m package versions... •••

[39]:
# Загрузка данных:
houses = CSV.File("houses.csv") |> DataFrame

985×12 DataFrame                                     960 rows omitted
  Row  street      city      zip  state  beds  baths  sq_ft  type      sale_date
   String      String15  Int64  String3  Int64  Int64  Int64  String15  String31
-----
  1  3526 HIGH ST  SACRAMENTO  95838  CA      2      1    836  Residential  Wed May 21 00:00:00 EDT 2008
  2  51 OMAHA CT  SACRAMENTO  95823  CA      3      1   1167  Residential  Wed May 21 00:00:00 EDT 2008
  3  2796 BRANCH ST  SACRAMENTO  95815  CA      2      1    796  Residential  Wed May 21 00:00:00 EDT 2008
```

Рис. 2.20: Кластеризация данных. Метод k-средних

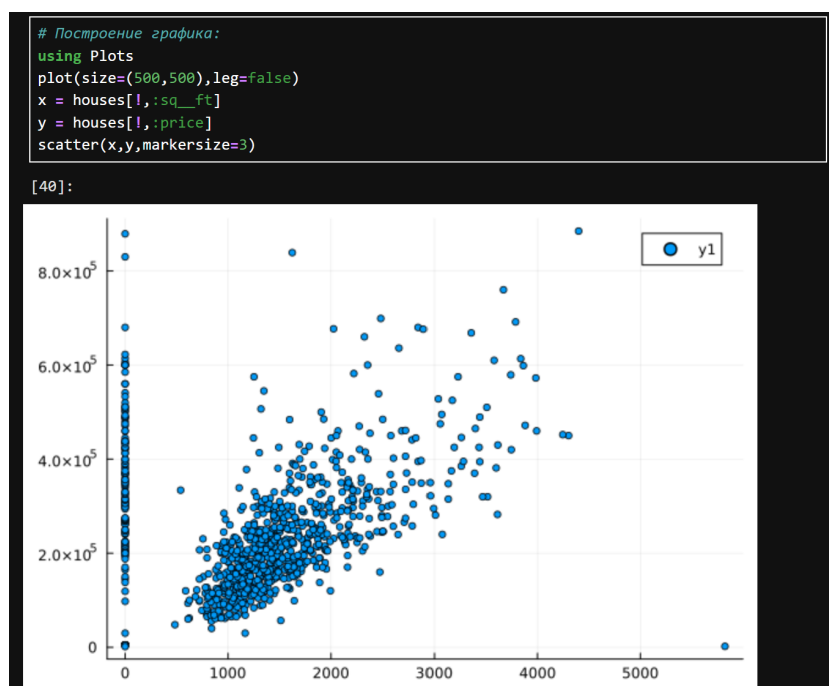


Рис. 2.21: Кластеризация данных. Метод k-средних

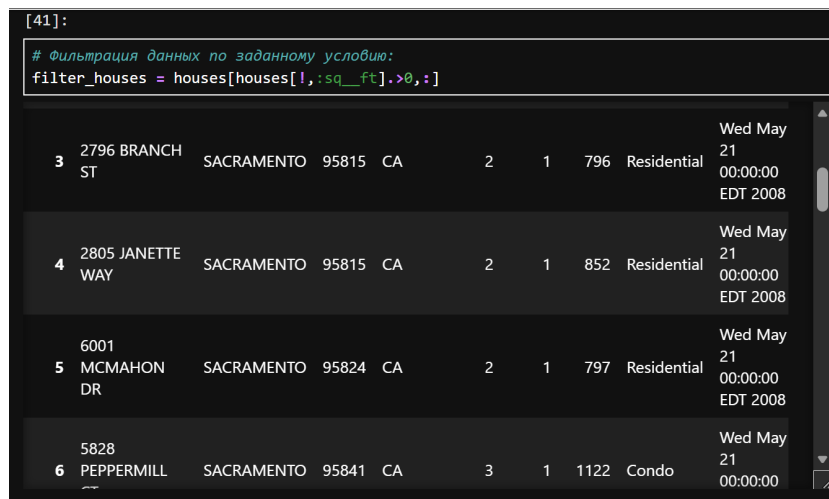


Рис. 2.22: Кластеризация данных. Метод k-средних

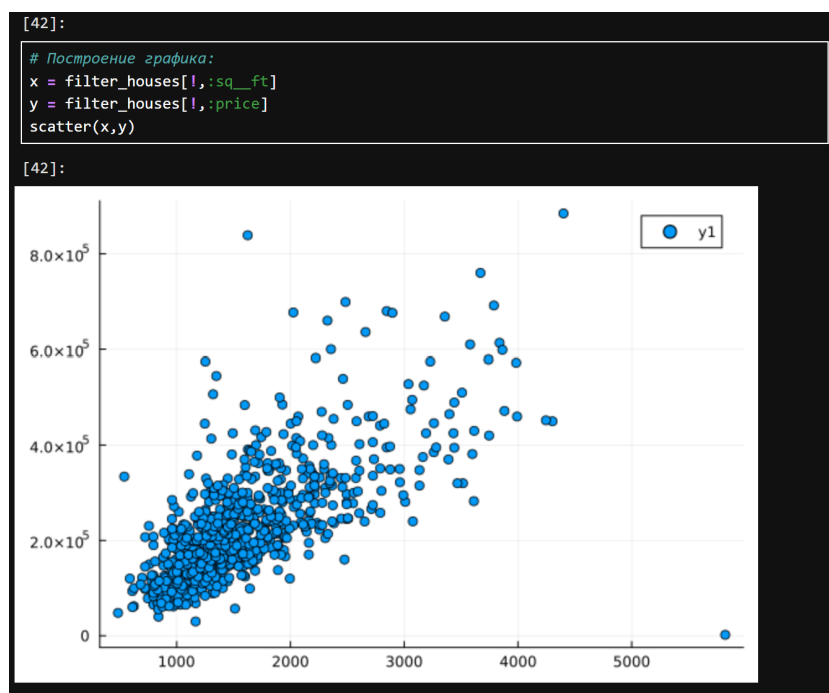


Рис. 2.23: Кластеризация данных. Метод k-средних

```

using Statistics
using DataFrames

result = combine(groupby(filter_houses, :type), :price => mean => :mean_price)

[43]:
3×2 DataFrame
  Row  type      mean_price
   String15      Float64
1  Residential  2.34802e5
2   Condo      1.34213e5
3 Multi-Family  2.24535e5

[44]:
import Pkg
Pkg.add("Clustering")
using Clustering

```

⌕[32m⌕[1m Resolving⌕[22m⌕[39m package versions... •••

Рис. 2.24: Кластеризация данных. Метод k-средних

```

[45]:
# Добавление данных :latitude и :longitude в новый фрейм:
X = filter_houses[:,[:latitude, :longitude]]
# Конвертация данных в матричный вид:
X = Matrix(X)

[45]:
814×2 Matrix{Float64}:
38.6319  -121.435
38.4789  -121.431
38.6183  -121.444
38.6168  -121.439
38.5195  -121.436
38.6626  -121.328
38.6817  -121.352
38.5351  -121.481
38.6212  -121.271
38.7009  -121.443
38.6377  -121.452
38.4707  -121.459
38.6187  -121.436
⋮
38.7035  -121.375
38.7031  -121.235
38.3898  -121.446
38.8978  -121.325
38.4679  -121.445
38.4453  -121.442
38.4174  -121.484
38.4577  -121.36
38.4999  -121.459
38.7088  -121.257
38.417   -121.397
38.6552  -121.076

```

Рис. 2.25: Кластеризация данных. Метод k-средних

```

X = X'

[46]:
2x814 adjoint(::Matrix{Float64}) with eltype Float64:
 38.6319  38.4789  38.6183  ...  38.7088  38.417  38.6552
-121.435 -121.431 -121.444  ... -121.257 -121.397 -121.076

[47]:
# Задание количества кластеров:
k = length(unique(filter_houses[:, :zip]))

[47]:
66

[48]:
# Определение k-среднего:
C = kmeans(X, k)

[48]:
KmeansResult{Matrix{Float64}, Float64, Int64}([38.4144165 38.45262856756756 ... 38.494276000
000006 38.247659; -121.19153449999999 -121.42909340540542 ... -121.41519773333336 -121.51512
9], [20, 58, 20, 20, 37, 54, 48, 49, 28, 44 ... 45, 53, 58, 2, 40, 52, 10, 55, 36, 11],
[6.185709571582265e-5, 0.00044116201024735346, 7.146993084461428e-5, 7.716179970884696e-5,
0.00017851325173978694, 0.00017962520360015333, 0.0002996578950842377, 4.6573350118706e-5,
0.0012155475087638479, 0.00011686658399412408 ... 1.1883959814440459e-5, 5.241325561655685
e-5, 9.496071652392857e-5, 0.00020711692559416406, 0.000914188327442389, 0.000138507319206
83756, 7.882461432018317e-5, 0.0005097794673929457, 0.00026846003675018437, 0.000665679428
4936041], [2, 37, 3, 18, 20, 10, 1, 17, 13, 10 ... 1, 25, 25, 4, 1, 26, 3, 19, 15, 1], [2,
37, 3, 18, 20, 10, 1, 17, 13, 10 ... 1, 25, 25, 4, 1, 26, 3, 19, 15, 1], 0.200064121658215
3, 10, true)

```

Рис. 2.26: Кластеризация данных. Метод k-средних

[49]:

Формирование фрейма данных:
df = DataFrame(cluster = C.assignments,city = filter_houses[:,city], latitude = filter_ho

[49]:

814x5 DataFrame

789 rows omitted

Row	cluster	city	latitude	longitude	zip
	Int64	String15	Float64	Float64	Int64
1	20	SACRAMENTO	38.6319	-121.435	95838
2	58	SACRAMENTO	38.4789	-121.431	95823
3	20	SACRAMENTO	38.6183	-121.444	95815
4	20	SACRAMENTO	38.6168	-121.439	95815
5	37	SACRAMENTO	38.5195	-121.436	95824
6	54	SACRAMENTO	38.6626	-121.328	95841
7	48	SACRAMENTO	38.6817	-121.352	95842
8	49	SACRAMENTO	38.5351	-121.481	95820
9	28	RANCHO CORDOVA	38.6212	-121.271	95670
10	44	RIO LINDA	38.7009	-121.443	95673
11	59	SACRAMENTO	38.6377	-121.452	95838
12	58	SACRAMENTO	38.4707	-121.459	95823
13	20	SACRAMENTO	38.6187	-121.436	95815
⋮	⋮	⋮	⋮	⋮	⋮
803	23	NORTH HIGHLANDS	38.7035	-121.375	95660
804	55	ORANGEVALE	38.7031	-121.235	95662

Рис. 2.27: Кластеризация данных. Метод k-средних

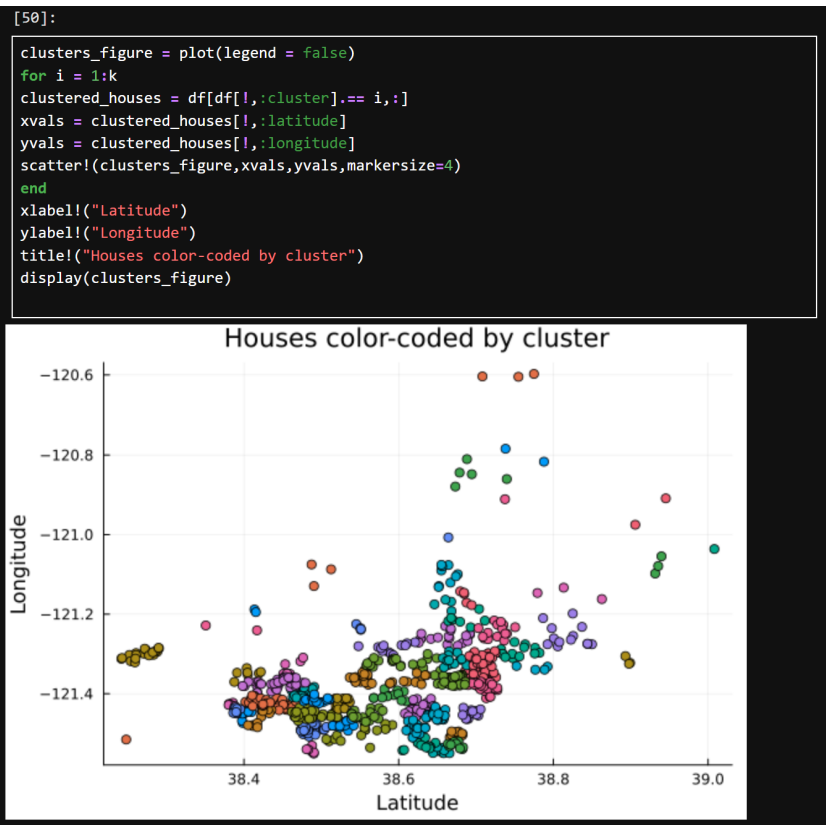


Рис. 2.28: Кластеризация данных. Метод k-средних

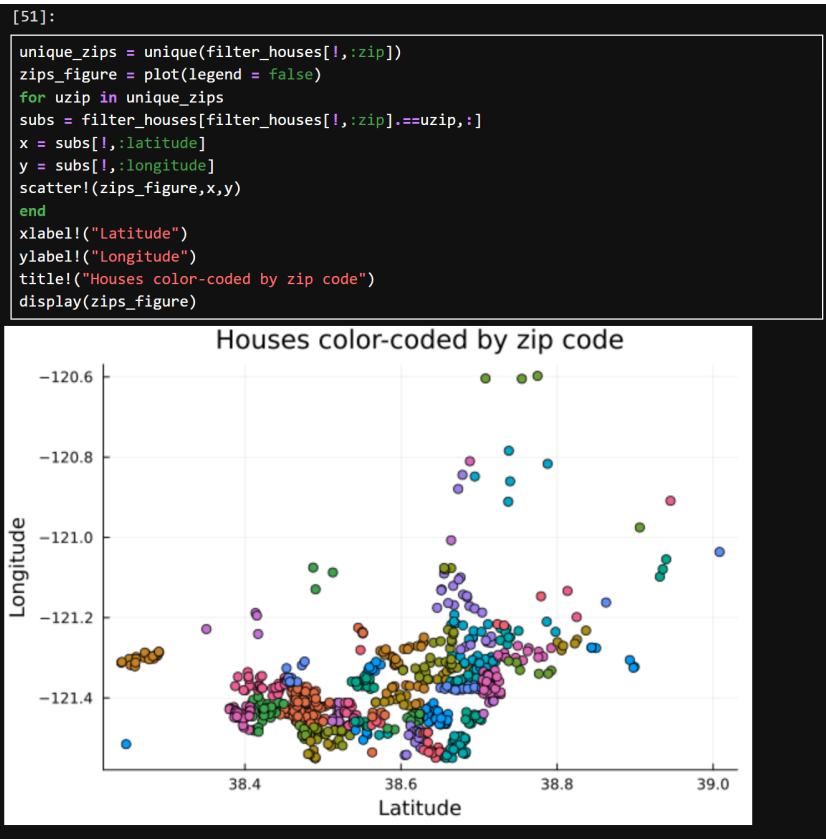


Рис. 2.29: Кластеризация данных. Метод k-средних

```
[52]:
# Подключение пакета NearestNeighbors:
import Pkg
Pkg.add("NearestNeighbors")
using NearestNeighbors

[32m[1m Resolving[22m[39m package versions...•••

[53]:
knearest = 10
id = 70
point = X[:,id]

[53]:
2-element Vector{Float64}:
 38.44004
-121.421012

[54]:
# Поиск ближайших соседей:
kdtree = KDTree(X)
idxs, dists = knn(kdtree, point, knearest, true)

[54]:
([70, 764, 196, 125, 557, 368, 415, 92, 112, 683], [0.0, 0.006264891539364138, 0.008253202
59050462, 0.008473585132630057, 0.009164073548370186, 0.009405065124697706, 0.009921759722
95076, 0.009941028618812013, 0.01033263770777167, 0.011168993911721985])
```

Рис. 2.30: Кластеризация данных. Метод k ближайших соседей

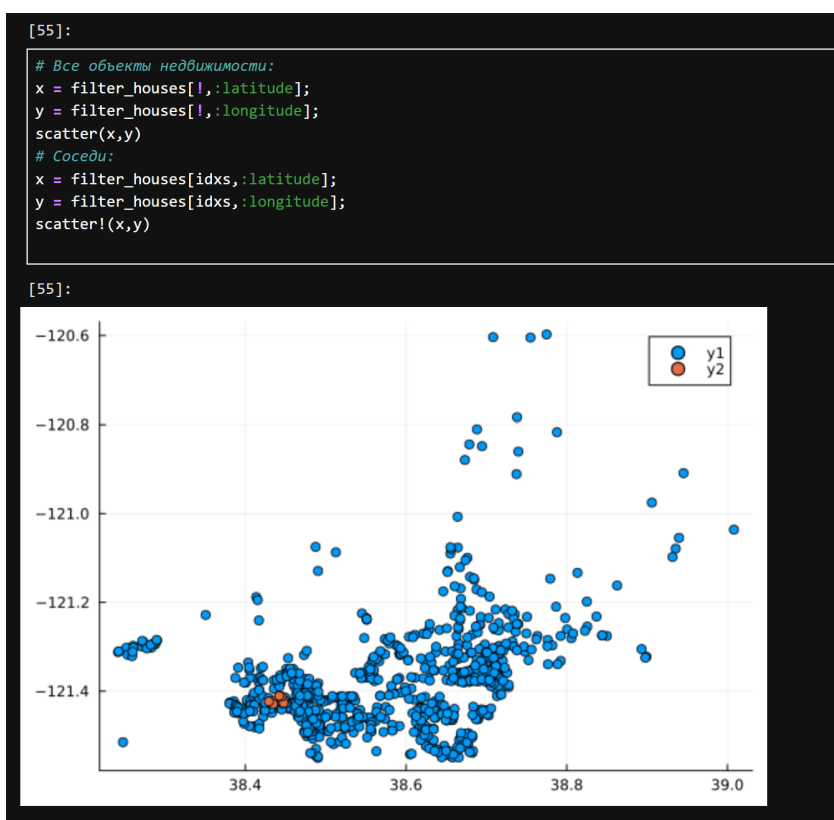


Рис. 2.31: Кластеризация данных. Метод k ближайших соседей

```
[56]:
# Фильтрация по районам соседних домов:
cities = filter_houses[idxs,:city]

[56]:
10-element PooledArrays.PooledVector{String15, UInt32, Vector{UInt32}}:
"SACRAMENTO"
"ELK GROVE"
"SACRAMENTO"
"SACRAMENTO"
"SACRAMENTO"
"SACRAMENTO"
"ELK GROVE"
"ELK GROVE"
"ELK GROVE"
"ELK GROVE"
```

Рис. 2.32: Кластеризация данных. Метод k ближайших соседей

```
[57]:
# Фрейм с указанием площади и цены недвижимости:
F = filter_houses[!, [:sq_ft, :price]]
y = filter_houses[!, :price];
# Конвертация данных в массив:
F = Array{F}'

[57]:
2×814 adjoint(::Matrix{Int64}) with eltype Int64:
 836  1167   796   852   797  1122  ...  1477   1216   1685   1362
59222 68212 68880 69307 81900 89921  ... 234000 235000 235301 235738

[58]:
# Подключение пакета MultivariateStats:
import Pkg
Pkg.add("MultivariateStats")
using MultivariateStats

[32m[1m Resolving[22m[39m package versions... •••
```

Рис. 2.33: Обработка данных. Метод главных компонент


```
[59]:
# Приведение типов данных к распределению для PCA:
M = fit(PCA, F)

[59]:
PCA(indim = 2, outdim = 1, principalratio = 0.9999840784692097)

Pattern matrix (unstandardized loadings):
```

	PC1
1	460.52
2	1.19826e5

```
Importance of components:
```

	PC1
SS Loadings (Eigenvalues)	1.43584e10
Variance explained	0.999984
Cumulative variance	0.999984
Proportion explained	1.0
Cumulative proportion	1.0

Рис. 2.34: Обработка данных. Метод главных компонент

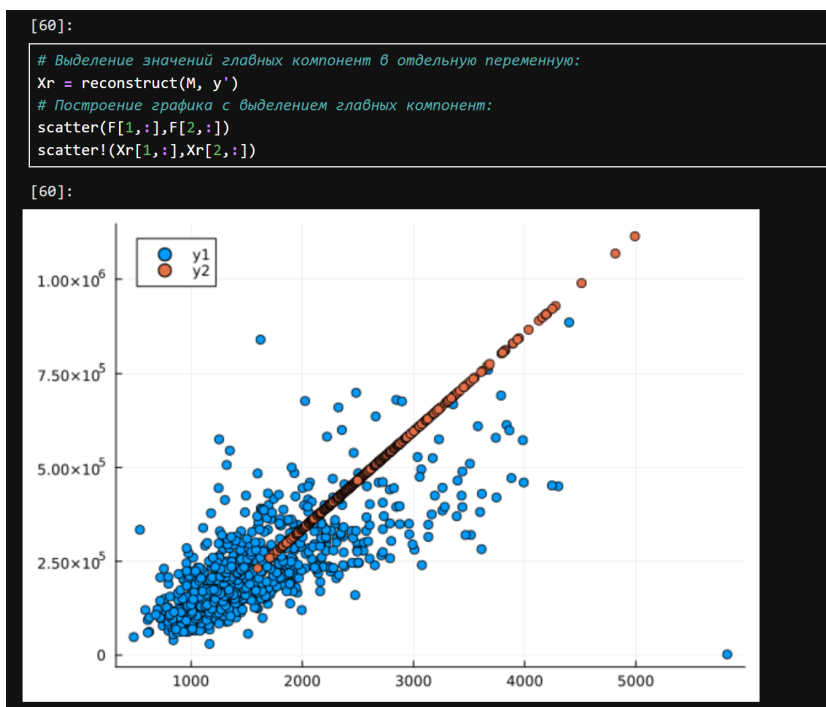


Рис. 2.35: Обработка данных. Метод главных компонент

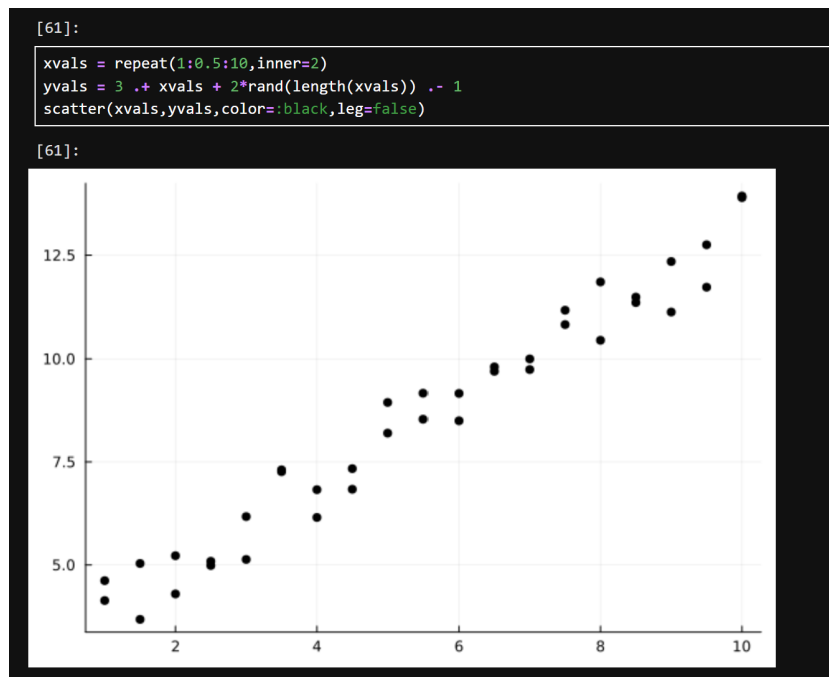


Рис. 2.36: Обработка данных. Линейная регрессия

```
function find_best_fit(xvals,yvals)
    meanx = mean(xvals)
    meany = mean(yvals)
    stdx = std(xvals)
    stdy = std(yvals)
    r = cor(xvals,yvals)
    a = r*stdy/stdx
    b = meany - a*meanx
    return a,b
end
```

[62]:

```
find_best_fit (generic function with 1 method)
```

Рис. 2.37: Обработка данных. Линейная регрессия

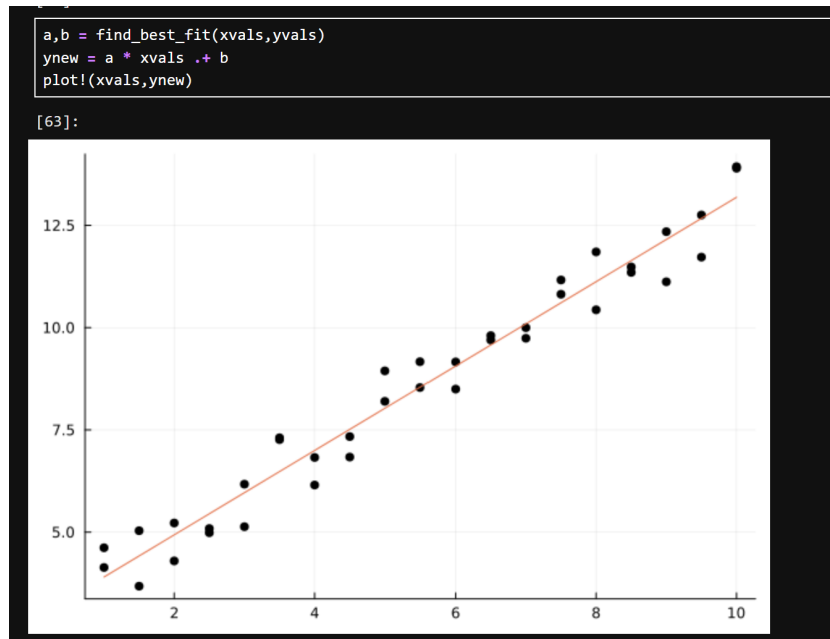


Рис. 2.38: Обработка данных. Линейная регрессия

```

xvals = 1:100000;
xvals = repeat(xvals,inner=3);
yvals = 3 .+ xvals + 2*rand(length(xvals)) .- 1;
@show size(xvals)
@show size(yvals)

size(xvals) = (300000,)
size(yvals) = (300000,)
[64]:

(300000,)

[65]:

@time a,b = find_best_fit(xvals,yvals)

0.050306 seconds (33.26 k allocations: 1.657 MiB, 95.82% compilation time)
[65]:

(0.9999999832029329, 3.000153184722876)

```

Рис. 2.39: Обработка данных. Линейная регрессия

Выполним задания для самостоятельного выполнения: (рис. 2.40 - 2.57).

Используем Clustering.jl для кластеризации на основе k-средних. Сделаем точечную диаграмму полученных кластеров. Для этого проиндексируем фрейм данных, преобразуем его в массив и транспонируем.

7.4.1. Кластеризация

Загрузите
`using RDatasets`
`iris = dataset("datasets", "iris")`
Используйте `Clustering.jl` для кластеризации на основе k-средних. Сделайте точечную диаграмму полученных кластеров.
Подсказка: вам нужно будет проиндексировать фрейм данных, преобразовать его в массив и транспонировать.

[74]:

```
using RDatasets

iris = dataset("datasets", "iris")
```

[74]:

150×5 DataFrame

125 rows omitted

Row	SepalLength	SepalWidth	PetalLength	PetalWidth	Species
	Float64	Float64	Float64	Float64	Cat...
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3.0	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
5	5.0	3.6	1.4	0.2	setosa
6	5.4	3.9	1.7	0.4	setosa
7	4.6	3.4	1.4	0.3	setosa
8	5.0	3.4	1.5	0.2	setosa
9	4.4	2.9	1.4	0.2	setosa
10	4.9	3.1	1.5	0.1	setosa

Рис. 2.40: Задание для самостоятельного выполнения №1

```
unique(iris[:,5])
```

[75]:

3-element CategoricalArrays.CategoricalArray{String,1,UInt8}:
"setosa"
"versicolor"
"virginica"

[76]:

```
iris_type = Dict{"setosa" => 1, "versicolor" => 2, "virginica" => 3}  
iris[:,5] = [iris_type[i] for i in iris[:,5]]  
X = Matrix(iris)
```

[76]:

5×150 adjoint(::Matrix{Float64}) with eltype Float64:
5.1 4.9 4.7 4.6 5.0 5.4 4.6 5.0 ... 6.8 6.7 6.7 6.3 6.5 6.2 5.9
3.5 3.0 3.2 3.1 3.6 3.9 3.4 3.4 ... 3.2 3.3 3.0 2.5 3.0 3.4 3.0
1.4 1.4 1.3 1.5 1.4 1.7 1.4 1.5 ... 5.9 5.7 5.2 5.0 5.2 5.4 5.1
0.2 0.2 0.2 0.2 0.2 0.4 0.3 0.2 ... 2.3 2.5 2.3 1.9 2.0 2.3 1.8
1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 ... 3.0 3.0 3.0 3.0 3.0 3.0 3.0

[77]:

```
k = length(unique(iris[:,5]))
```

[77]:

3

Рис. 2.41: Задание для самостоятельного выполнения №1

```
C = kmeans(X, k)
```

```
[78]:
```

```
KmeansResult{Matrix{Float64}, Float64, Int64}([5.005999999999999 6.6224489795918355 5.9156
86274509803; 3.428000000000001 2.983673469387755 2.7647058823529416; ... ; 0.245999999999999
9 2.0326530612244893 1.3333333333333333; 1.0 3.0 2.019607843137255], [1, 1, 1, 1, 1, 1,
1, 1, 1 ... 2, 2, 2, 2, 2, 2, 2, 2, 2], [0.019980000000018094, 0.20038000000000977, 0.1
7398000000001446, 0.2759800000000041, 0.03557999999999595, 0.45838000000000534, 0.17238000
000000397, 0.00438000000011819, 0.6519799999999947, 0.1415800000000189 ... 0.155193669304
48086, 0.38621407746771297, 0.9986630570595594, 0.25641815910034893, 0.3404997917534729,
0.21723448563099623, 0.6843773427738427, 0.15580591420243195, 0.4533569346105537, 0.800499
791753424], [50, 49, 51], [50, 49, 51], 87.22062785114079, 10, true)
```

```
[80]:
```

```
df = DataFrame(cluster = C.assignments, SepalLength = iris[:,SepalLength],
SepalWidth = iris[:,SepalWidth], PetalLength = iris[:,PetalLength],
PetalWidth = iris[:,PetalWidth], Species = iris[:,Species])
```

```
[80]:
```

150×6 DataFrame 125 rows omitted

Row	cluster	SepalLength	SepalWidth	PetalLength	PetalWidth	Species
	Int64	Float64	Float64	Float64	Float64	Int64
1	1	5.1	3.5	1.4	0.2	1
2	1	4.9	3.0	1.4	0.2	1
3	1	4.7	3.2	1.3	0.2	1
4	1	4.6	3.1	1.5	0.2	1
5	1	5.0	3.6	1.4	0.2	1
6	1	5.4	3.9	1.7	0.4	1
7	1	4.6	3.4	1.4	0.3	1
8	1	5.0	3.4	1.5	0.2	1

Рис. 2.42: Задание для самостоятельного выполнения №1

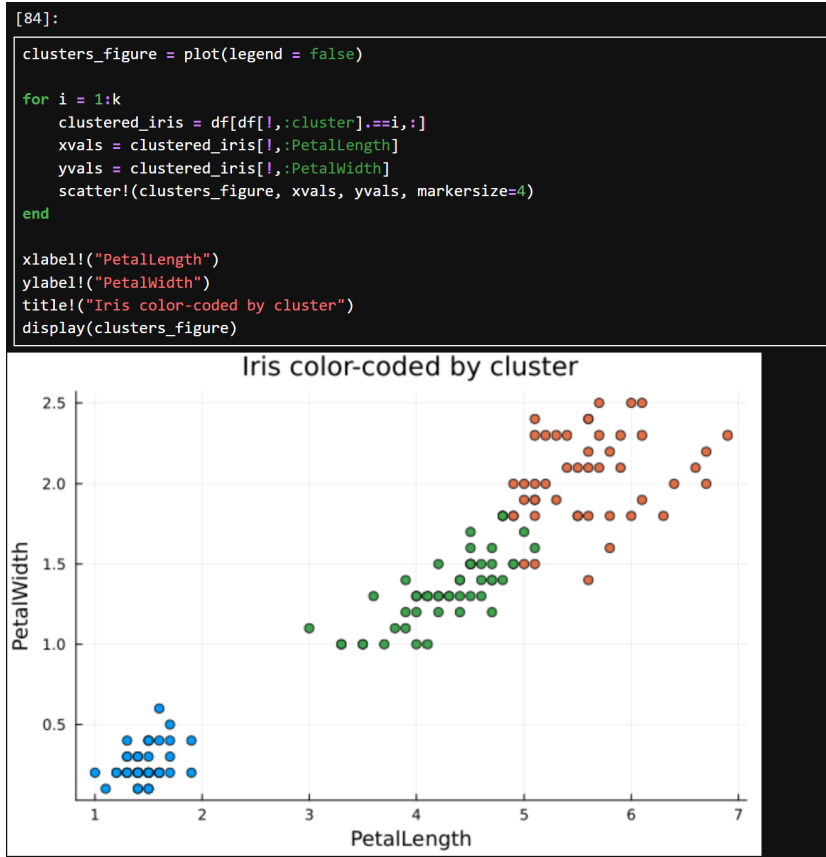


Рис. 2.43: Задание для самостоятельного выполнения №1

Создадим матрицу данных X_2 , которая добавляет столбец единиц в начало матрицы данных, и решим систему линейных уравнений.

Часть 1 Пусть регрессионная зависимость является линейной. Матрица наблюдений факторов X имеет размерность $N \times 3$ `randn(N, 3)`, массив результатов $N \times 1$, регрессионная зависимость является линейной. Найдите МНК-оценку для линейной модели.

- Сравните свои результаты с результатами использования `llsq` из `MultivariateStats.jl` (посмотрите документацию).
- Сравните свои результаты с результатами использования регулярной регрессии наименьших квадратов из `GLM.jl`.

Подсказка. Создайте матрицу данных X_2 , которая добавляет столбец единиц в начало матрицы данных, и решите систему линейных уравнений. Объясните с помощью теоретических выкладок.

```
[91]:
X = randn(1000, 3)
a0 = rand(3)
y = X * a0 + 0.1 * randn(1000);
```

Рис. 2.44: Задание для самостоятельного выполнения №2

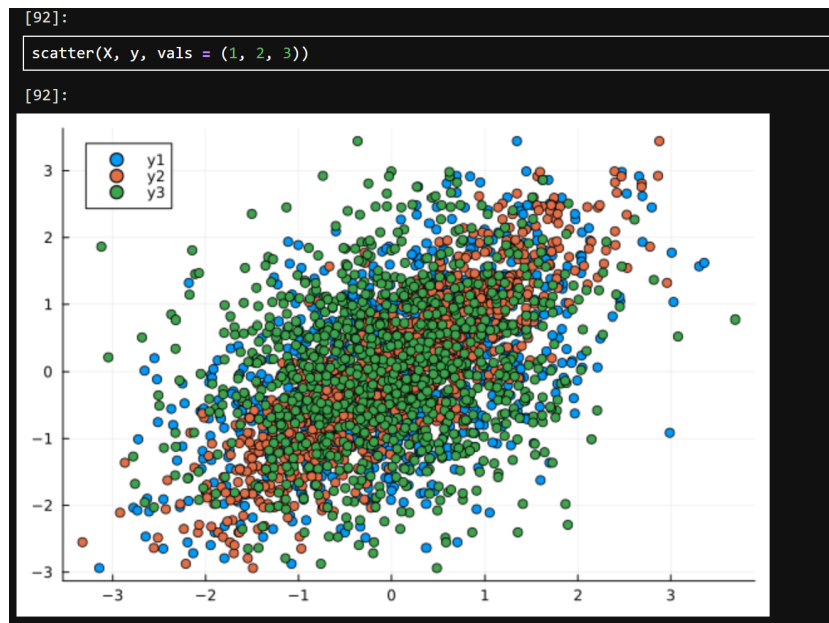


Рис. 2.45: Задание для самостоятельного выполнения №2

```
X2 = hcat(ones(1000), X);
beta = (X2' * X2) \ (X2' * y)
```

[93]:

```
4-element Vector{Float64}:
-0.003194306445050312
 0.5532633083681374
 0.9282068296353732
 0.22616432687440316
```

[94]:

```
using MultivariateStats
beta_mvs = llsq(X, y)
```

[94]:

```
4-element Vector{Float64}:
 0.5532633083681374
 0.9282068296353733
 0.2261643268744032
-0.0031943064450503118
```

Рис. 2.46: Задание для самостоятельного выполнения №2

```
[95]:
using GLM

df = DataFrame(hcat(X, y), :auto)
rename!(df, ["x1", "x2", "x3", "y"])
model = lm(@formula(y ~ x1 + x2 + x3), df)
coef(model)

[95]:
4-element Vector{Float64}:
-0.003194306445050314
 0.5532633083681374
 0.9282068296353732
 0.22616432687440313
```

Рис. 2.47: Задание для самостоятельного выполнения №2

Часть 2 Найдите линию регрессии, используя данные (X, y) . Постройте график (X, y) , используя точечный график. Добавьте линию регрессии, используя `abline!`. Добавьте заголовок «График регрессии» и подпишите оси x и y .

```
[96]:
X = rand(100);
y = 2X + 0.1 * randn(100);

[97]:
a, b = find_best_fit(X, y)

[97]:
(2.0452706570443224, -0.040425431114616606)
```

Рис. 2.48: Задание для самостоятельного выполнения №2

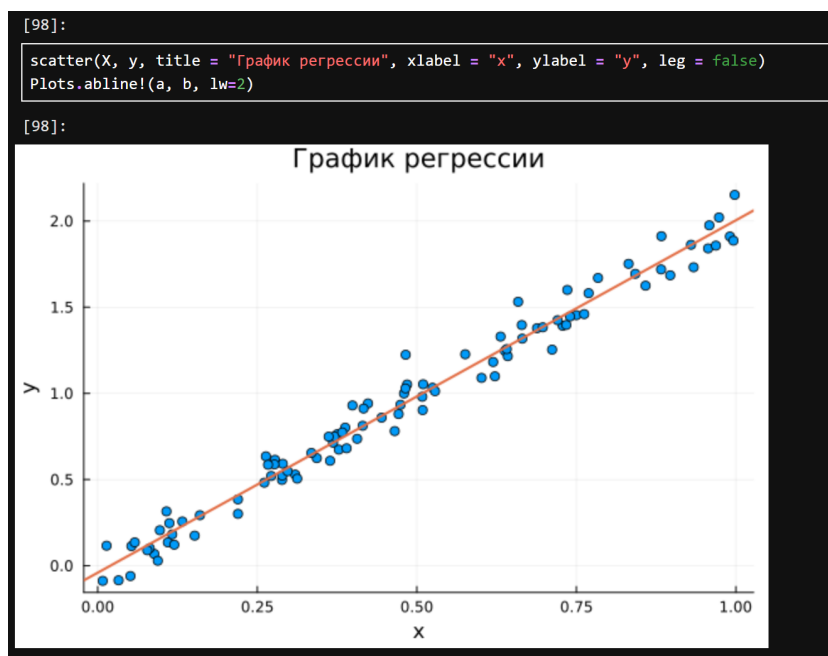


Рис. 2.49: Задание для самостоятельного выполнения №2

7.4.3. Модель ценообразования биномиальных опционов

Описание модели ценообразования биномиальных опционов можно найти на стр. https://en.wikipedia.org/wiki/Binomial_options_pricing_model.

Постройте траекторию возможных цен на акции:

- S — начальная цена акции;
- T — длина биномиального дерева в годах;
- n — количество периодов;
- $h = T/n$ — длина одного периода;
- σ — волатильность акции;
- r — годовая процентная ставка;
- $u = \exp(rh + \sigma\sqrt{h})$;
- $d = \exp(rh - \sigma\sqrt{h})$;
- $p^* = \frac{\exp(rh) - d}{u - d}$.

- a) Пусть $S = 100$, $T = 1$, $n = 10000$, $\sigma = 0.3$ и $r = 0.08$. Попробуйте построить траекторию курса акций. Функция `rand()` генерирует случайное число от 0 до 1. Вы можете использовать функцию построения графика из библиотеки графиков.
- b) Создайте функцию `createPath(S :: Float64, r :: Float64, sigma :: Float64, T :: Float64, n :: Int64)`, которая создает траекторию цены акции с учетом начальных параметров. Используйте `createPath`, чтобы создать 10 разных траекторий и построить их все на одном графике.
- c) Распараллельте генерацию траекторий. Можете использовать `Threads.@threads`, `rtask` и `@parallel`.
- d) Пусть $S = 100$, $T = 1$, $n = 10000$, $\sigma = 0.3$ и $r = 0.08$. Попробуйте построить траекторию курса акций. Функция `rand()` генерирует случайное число от 0 до 1. Вы можете использовать функцию построения графика из библиотеки графиков.

[166]:

```
S = 100.0
T = 1.0
n = 10000
sigma = 0.3
r = 0.08;
```

Рис. 2.50: Задание для самостоятельного выполнения №3

Пусть $S = 100$, $T = 1$, $n = 10000$, $\sigma = 0.3$ и $r = 0.08$. Попробуем построить траекторию курса акций.

```
[159]:
h = T/n
u = exp(r * h + sigma * sqrt(h))
d = exp(r * h - sigma * sqrt(h))
p = (exp(r * h) - d) / (u - d);

[175]:
# a)
function createPath(S, r, sigma, T, n)
    path = []
    S_ = S
    push!(path, S)
    for i in 1:n
        if rand() < p
            S_ *= u
        else
            S_ *= d
        end
        push!(path, S_)
    end
    return path
end

[175]:
createPath (generic function with 1 method)
```

Рис. 2.51: Задание для самостоятельного выполнения №3

```
path = createPath(S, r, sigma, T, n)

[164]:

10001-element Vector{Any}:
 100
100.30125285715093
100.60341324714126
100.30285769003527
100.60502291463052
100.9080984205982
100.60663260787467
100.30606743283697
100.60824232687412
100.91132753141315
101.21532578879047
100.91294212557038
101.21694524695872
```

Рис. 2.52: Задание для самостоятельного выполнения №3



Рис. 2.53: Задание для самостоятельного выполнения №3

Создадим функцию `createPath(S :: Float64, r :: Float64, sigma :: Float64, T :: Float64, n :: Int64)`, которая создает траекторию цены акции с учетом начальных параметров. Используем `createPath`, чтобы создать 10 разных траекторий и построим их все на одном графике

```
# b)
function createPath2(S::Float64, r::Float64, sigma::Float64, T::Float64, n::Int64)
    Y = Vector{Float64}()
    X = collect(0:T/n:1)
    h = T/n
    u = exp(r * h + sigma * sqrt(h))
    d = exp(r * h - sigma * sqrt(h))
    p = (exp(r * h) - d) / (u - d)
    for i=0:n
        prob = rand()
        if prob < p
            S *= u
        else
            S *= d
        end
        push!(Y, S)
    end
    return (X, Y)
end

[176]:
createPath2 (generic function with 1 method)
```

Рис. 2.54: Задание для самостоятельного выполнения №3

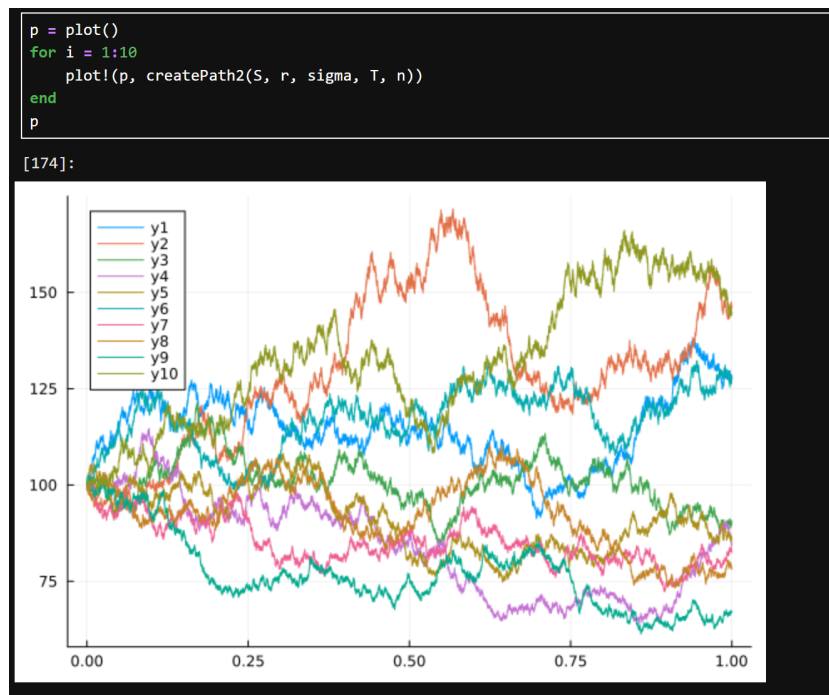


Рис. 2.55: Задание для самостоятельного выполнения №3

Распараллелим генерацию траектории. Можем использовать `Threads.@threads`, `pmap` и `@parallel`

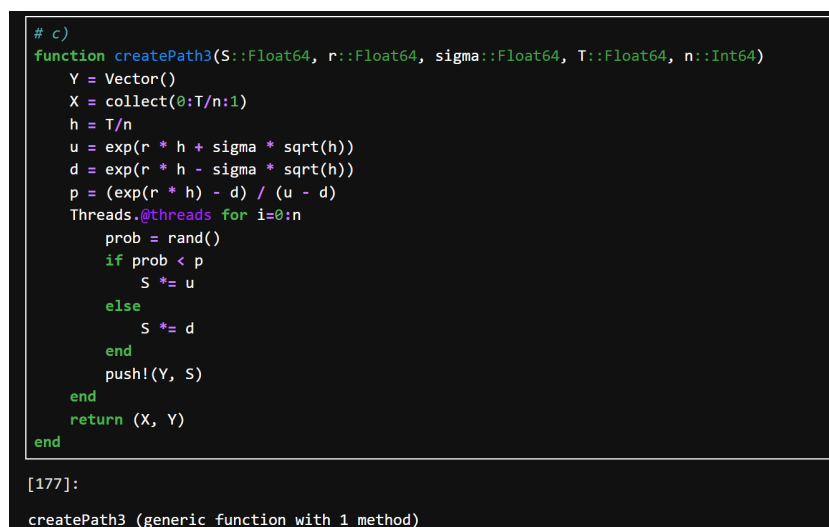


Рис. 2.56: Задание для самостоятельного выполнения №3

[178]:

```
p = plot()
for i = 1:10
    plot!(p, createPath3(S, r, sigma, T, n))
end
p
```

[178]:

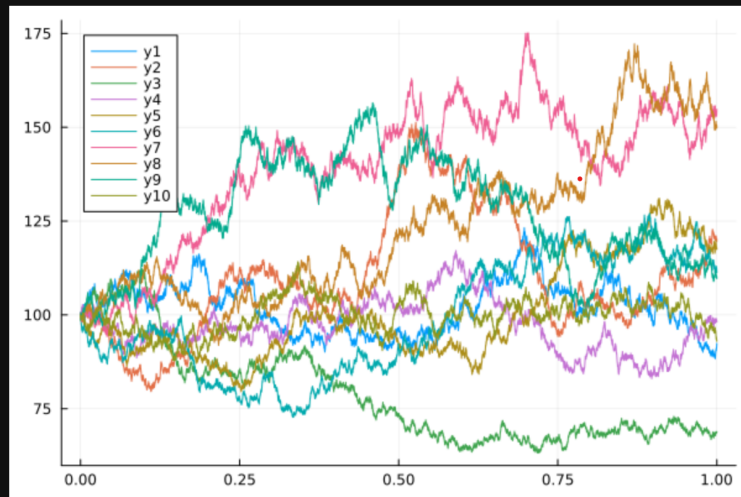


Рис. 2.57: Задание для самостоятельного выполнения №3

3 Выводы

В результате выполнения работы были освоены специализированные пакеты Julia для обработки данных.